The AGH University of Krakow

Project documentation

Airline Reservation System

Object-oriented programming languages

Electronics 2nd year

Author:

Norbert Bukała

23.01.2025

1. Project description

Airline Reservation System is a console application written in C++ that allows users to manage flight reservations. The system offers two main modes: client and administration, making it suitable for both passengers and flight management staff. The project demonstrates the practical application of an object-oriented approach to programming, file manipulation, and data validation.

2. User Manual

At the start of the program, the user selects a role – customer or admin. If the first option is selected, three options appear: 1. View Flights – showing information about current flights, 2. Book a Flight – the ability to book a flight, 3. Exit – exiting the program. The rest of the program revolves around the second option, after which the user is asked to enter the flight number they are interested in. The program then waits for the customer's details (name, surname, ID, age, address) and whether they wish to purchase an additional baggage allowance. In the final section, the user selects a seat by entering a number (row) and a letter (column). After this, a summary of the reservation is displayed (flight selection, provided details, selected seat, and total cost) and a final question about whether to maintain the reservation, which the user can accept with "1" or decline with "2." After confirmation, the user's data is transferred to the FLxxx_passengers.txt file, and the reserved seat is marked with an "X" in the FLxxx_seats.txt file.

After selecting the admin option, the user is prompted to enter a password for security reasons. Next, in addition to the previously mentioned "Exit" option, the option to delete a reservation appears. This is done by selecting the flight number (e.g., FL101). After selecting the desired flight, the admin specifies the seat from which the reservation should be deleted. Once the admin follows the above instructions, the customer data will be deleted from the FLxxx_passengers.txt file, and the reserved seat "X" will be replaced with "O" for the selected flight in the FLxxx_seats.txt file.

3. Compilation

To compile the program, simply use a standard C++ compiler. The project runs on operating systems that support the standard C++ compiler, such as Windows, Linux, and macOS.

Compiling with MinGW:

1. Open cmd (command prompt) or PowerShell.

2. Navigate to the directory containing the source files: cd C:\path\to\project.

3. Enter the command: g++ main.cpp -o main

4. Run the compiled program: ./main.

4. Source files

In this section, you should describe all the source files (.cpp, .h) in the project. Provide the name of each file and information about its contents. For example:

The project consists of the following source files:

- Flight.hpp – declaration and implementation of the Flight class,

- Passenger.hpp – declaration and implementation of the Passenger class,

 SeatReservation.hpp – declaration and implementation of the SeatReservation class,

 Utilities.hpp – declaration and implementation of various useful functions.

 main.cpp – contains the main program function and two functions: void clientMenu() and void adminMenu(). The clientMenu function is responsible for the user interface for clients: it allows them to browse available flights, reserve seats, and enter passenger data (e.g., first name, last name, and baggage). The adminMenu function is used by the administrator to manage reservations: it allows them to delete occupied seats (e.g., in the event of errors or cancellations) and update data files. Both menus control the program's business logic, separating client functions from administrative privileges.

5. Class Description

This section should include a description of all classes created in the project. You should also include the purpose of each class and information about its public methods. Optionally, you can include source code fragments. For example:

W projekcie utworzono następujące klasy:

- Flight – represents basic information about a given flight.
  - void displayFlight()const – displays flight information in an orderly manner.

```
void displayFlight() const {
    std::cout << std::setw(10) << flightNumber << " | "
              << std::setw(15) << departure << " -> "
              << std::setw(15) << destination << " | Departure: "
              << departureTime << " | Arrival: " << arrivalTime << " | Ticket cost: " << ticketCost << "\n";
}
```

- Passenger – represents basic information about a given passenger.
  - std::string toString() const – Converts an object to a string containing its data in a readable form. The function creates a string containing information about the first name, last name, identification number (ID), age, address, extra baggage, and seat number, and then returns the string.

```
std::string toString() const {
    return name + " " + surname + ", ID: " + idNumber + ", Age: " + std::to_string(age) + ", Address: "
    + address + ", Extra Luggage: " + (extraLuggage ? "Yes" : "No") + ", Seat: " + std::to_string(seatRow) + seatCol;
}
```

- SeatReservation – a class containing public and static methods and variables.
  - void displaySeats()const – displays the seating arrangement graphically on the console.

```cpp
void displaySeats() const {
    std::cout << "    ";
    for (size_t col = 0; col < seatLayout[0].size(); ++col) {
        std::cout << static_cast<char>('A' + col) << " ";
        if (col == 2) std::cout << "   ";
    }
    std::cout << "\n";

    for (size_t row = 0; row < seatLayout.size(); ++row) {
        std::cout << std::setw(2) << (row + 1) << " ";

        for (size_t col = 0; col < seatLayout[row].size(); ++col) {
            std::cout << "|";
            std::cout << seatLayout[row][col];
            if (col == 2) std::cout << "|  ";
        }
        std::cout << "|\n";
    }

    std::cout << "\nSeat Prices:\n";
    std::cout << "A, F: $10\nB, E: $8\nC, D: $4\n";
}
```

- bool reserveSeat(int row, int col) – used to reserve a seat based on the given row and column number. Checks whether the given indices fall within the seating array boundaries. If not, it throws a std::out_of_range exception. If the seat is already reserved (marked as 'X'), the function returns false. If the seat is available (marked as 'O'), it reserves it (changes it to 'X') and returns true.

```cpp
bool reserveSeat(int row, int col) {
    if (row < 0 || row >= (int)seatLayout.size() || col < 0 || col >= (int)seatLayout[row].size()) {
        throw std::out_of_range("Invalid seat position.");
    }
    if (seatLayout[row][col] == 'X') {
        return false;
    }
    seatLayout[row][col] = 'X';
    return true;
}
```

- bool removeSeat(int row, int col) – used to cancel a seat reservation based on the given row and column number. Checks whether the given indices are within the seating array boundaries. If not, it throws a std::out_of_range exception. If the seat is already available (marked as 'O'), the function returns false. If the seat is reserved (marked as 'X'), it removes the reservation (changes it to 'O') and returns true.

```cpp
bool removeSeat(int row, int col) {
    if (row < 0 || row >= (int)seatLayout.size() || col < 0 || col >= (int)seatLayout[row].size()) {
        throw std::out_of_range("Invalid seat position.");
    }
    if (seatLayout[row][col] == 'O') {
        return false;
    }
    seatLayout[row][col] = 'O';
    return true;
}
```

- void saveSeatsForFlight(const std::string& flightNumber) const – saves the state of the seating arrangement to a file named after the flight number (e.g., FL139_seats.txt). Opens the file and saves the state of each seat in each row, where 'O' indicates an available seat and 'X' indicates a reserved seat.

```
void saveSeatsForFlight(const std::string& flightNumber) const {
    std::string fileName = flightNumber + "_seats.txt";
    std::ofstream file(fileName);
    for (const auto& row : seatLayout) {
        for (char seat : row) {
            file << seat;
        }
        file << std::endl;
    }
}
```

- void loadSeatsForFlight(const std::string& flightNumber) – Loads the seat state from a file named after the flight number (e.g., FL123_seats.txt). Reads the file line by line and reconstructs vectors representing the seat state in the system. If the file is empty or contains invalid data, it throws a std::runtime_error exception.

```
void loadSeatsForFlight(const std::string& flightNumber) {
    std::string fileName = flightNumber + "_seats.txt";
    std::ifstream file(fileName);
    std::string line;

    seatLayout.clear();
    while (std::getline(file, line)) {
        seatLayout.emplace_back(line.begin(), line.end());
    }

    if (seatLayout.empty()) {
        throw std::runtime_error("Seat file is empty or invalid.");
    }
}
```

- Utilities – a class containing various useful functions.

  bool isValidName(const std::string& name) – Checks whether the given name is valid. Uses a regular expression (std::regex) that allows names consisting of letters (both uppercase and lowercase), dashes, and spaces. The function returns true if the name matches the pattern, otherwise false.

  bool isValidAge(int age) – Checks whether the age is within the allowed range. The function returns true if the age is greater than or equal to 0 and less than or equal to 130. Otherwise, it returns false.

  bool isValidId(const std::string& id) – Checks whether the given identifier (e.g., passport or ID number) is valid. Uses a regular expression that accepts a string consisting only of letters and numbers (no special characters). The function returns true if the identifier matches the pattern, otherwise false. bool getValidAge(int &age) – Prompts the user to enter their age and checks if it is valid (in the range 0-130). Uses a while loop to repeatedly enter their age if the user enters an invalid value (e.g., text or a number out of range). If the age is valid, it returns true and terminates the loop if it does not prompt the user to re-enter a value.

  bool isValidFlightNumber(const std::string& flightNum, const std::vector<Flight>& flights) – Checks if the provided flight number (flightNum) is valid, meaning it exists among the available flights. Uses the std::any_of function to check if at least one flight in the flights vector has a number corresponding to the provided flightNum. The function returns true if the flight number is found among the available flights, false otherwise. std::vector<Flight> loadFlights(const std::string&

fileName) – Loads flight data from a text file named fileName. For each line in the file (where the data is separated by spaces), the function creates a Flight object with the appropriate information (flight number, origin, destination, departure time, arrival time, ticket cost) and adds it to the flights vector. Finally, the function returns a vector of Flight objects.

 void savePassengerDetails(const std::string& flightNumber, const Passenger& passenger) – Saves passenger details to a text file whose name depends on the flight number (e.g., FL123_passengers.txt). Opens the file in append mode (std::ios::app), meaning that new information will be appended to the end of the file. Saves passenger information in text form using the toString() method of the Passenger object. If the file fails to open, it throws a std::runtime_error exception.

 void removePassengerDetails(const std::string& flightNumber, int row, char col) – removes information about the passenger who occupied a specific seat from a text file. Opens the passenger information file (flightNumber + "_passengers.txt") for reading and creates a temporary file (temp.txt). It loops through each line of the file and checks whether it contains information about the reserved seat (e.g., row number and column, e.g., "12B"). If it does, that line is skipped (i.e., deleted). After reading the file, it closes both files (original and temporary), then deletes the original file and renames the temporary file to the original passenger file name. This removes the line containing the passenger data for the specific seat.

```
bool isValidName(const std::string& name) {
    std::regex pattern("^[A-Za-z\\-\\s]+$");
    return std::regex_match(name, pattern);
}

bool isValidAge(int age) {
    return age >= 0 && age <= 130;
}

bool isValidId(const std::string& id) {
    std::regex pattern("^[A-Za-z0-9]+$");
    return std::regex_match(id, pattern);
}

bool getValidAge(int &age) {
    while (true) {
        std::cin >> age;
        if (std::cin.fail() || !isValidAge(age)) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid age. Please enter a valid age (0-130): ";
        } else {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            return true;
        }
    }
}


bool isValidFlightNumber(const std::string& flightNum, const std::vector<Flight>& flights) {
    return std::any_of(flights.begin(), flights.end(), [&flightNum](const Flight& flight) {
        return flight.flightNumber == flightNum;
    });
}
```

```cpp
std::vector<Flight> loadFlights(const std::string& fileName) {
    std::vector<Flight> flights;
    std::ifstream file(fileName);
    std::string num, dep, dest, dtime, atime, tcost;
    while (file >> num >> dep >> dest >> dtime >> atime >> tcost) {
        flights.emplace_back(num, dep, dest, dtime, atime, tcost);
    }
    return flights;
}

void savePassengerDetails(const std::string& flightNumber, const Passenger& passenger) {
    std::string fileName = flightNumber + "_passengers.txt";
    std::ofstream file(fileName, std::ios::app);
    if (file.is_open()) {
        file << passenger.toString() << std::endl;
    } else {
        throw std::runtime_error("Failed to open passenger file.");
    }
}

void removePassengerDetails(const std::string& flightNumber, int row, char col) {
    std::string fileName = flightNumber + "_passengers.txt";
    std::ifstream inputFile(fileName);
    std::ofstream tempFile("temp.txt");

    std::string line;
    while (std::getline(inputFile, line)) {
        if (line.find(std::to_string(row) + col) != std::string::npos) {
            continue;
        }
        tempFile << line << std::endl;
    }

    inputFile.close();
    tempFile.close();

    std::remove(fileName.c_str());
    std::rename("temp.txt", fileName.c_str());
}
```

6. Resources

The project uses the following resource files:

flights.txt – a file containing bank customer data. File structure:

- Flight number (e.g., FL139),
- Departure location,
- Arrival location,
- Departure time,
- Arrival time,
- Ticket cost.

FL[xxx]_seats.txt – a file containing bank customer data. File structure:

- O – seat available,
- X – seat occupied.

FL[xxx]_passengers.txt – a file containing customer data by flight. File structure:

- First name (separated by a space from the next data item),
- Last name (separated by a comma from the next data item),
- ID number (separated by a comma from the next data item),

- Age (separated by a comma from the next data item),
- Address (separated by a comma from the next data item),
- Purchase of additional baggage (separated by a comma from the next data item, YES – purchase, No – not using the service),
- Seat location on the plane (number – row, letter – column).

7. Further development and improvements

Customer functionality enhancements:

- Payment system: Integration with external payment systems (e.g., PayPal, Przelewy24) to process ticket payments.
- Booking history: Customers can view previous bookings, with the option to cancel or modify them.
- Notifications: Sending emails or text messages with booking confirmations, flight reminders, and information about changes.

Technical improvements:

- Database: Migration from text files to a database (e.g., SQLite, MySQL) for improved performance and data management.
- Graphical interface (GUI): Development of a more advanced graphical interface using libraries such as Qt or wxWidgets.
- Data validation: Adding more advanced mechanisms for validating user-entered data.

8. Others

Adding flights is done by creating new .txt files, which is only possible for the admin.